

The codebase is stronger than it looks. The stylesheet is where it's losing.

An evidence-based review of the Next.js 15 application at `D:\Websites\Vimtra Chennai Lions Gc Web` — 39 routes, 8 Prisma models, 13 server-action modules. The data layer is production-grade. Two things stand between this and launch: an order pipeline with no fulfilment end, and a design system that has been layered four times without a demolition.

AUDITED	BRANCH	ROUTES	TYPECHECK	LINT
3 Sep 2026	main	39	Pass	Clean

4	9	8,435	63 MB
P0 — BLOCKS PRODUCTION	P1 — HIGH PRIORITY	LINES IN GLOBALS.CSS	SHIPPED IN /PUBLIC
100%	0		
ADMIN ACTIONS AUTHORIZED	FABRICATED DATA POINTS FOUND		

A Executive summary

What this codebase actually is, after tracing it rather than assuming.

This is not the half-finished migration `CLAUDE.md` describes. The doc says "18 pages ported, news editor pending, orders not started." The repository actually contains **39 routes, a TipTap news editor, a full commerce schema with an atomic order-placement transaction, an inventory manager, and admin CRUD for fixtures, scores, standings and press coverage**. The documentation is roughly two milestones behind the code — which is itself a finding, because it is the map any new engineer will navigate by.

What is genuinely good

The order transaction is written the way it should be

STRENGTH

`placeOrder()` re-reads authoritative `Product` rows inside the transaction, never trusts client prices, decrements stock with a conditional `updateMany({ where: { stock: { gte: qty } } })` so a lost update can't oversell, freezes the shipping address as JSON and snapshots line items. This is careful, senior work.

lib/orders.ts:98-260

The data-integrity rule is being honoured in practice, not just in the doc

STRENGTH

Every number on `/invest`, `/partners`, `/the-club` and `data/players.ts` carries a brochure page citation in a comment. `/scores` and `/leaderboards` render honest unlit empty states instead of illustrative rows. I looked for invented statistics and found none.

app/invest/page.tsx:23; app/partners/page.tsx:28-29; data/players.ts:23-60

Authorization is server-side and complete

STRENGTH

All 19 admin server actions call `requireAdmin()` as their first statement. `middleware.ts` is correctly described in its own comment as a *coarse* cookie pre-check, not the real gate. Sessions are opaque random tokens in a DB table behind an `httpOnly` cookie — revocable, unlike a JWT.

lib/auth.ts:107-126; verified across app/admin/*/actions.ts

What is actually blocking

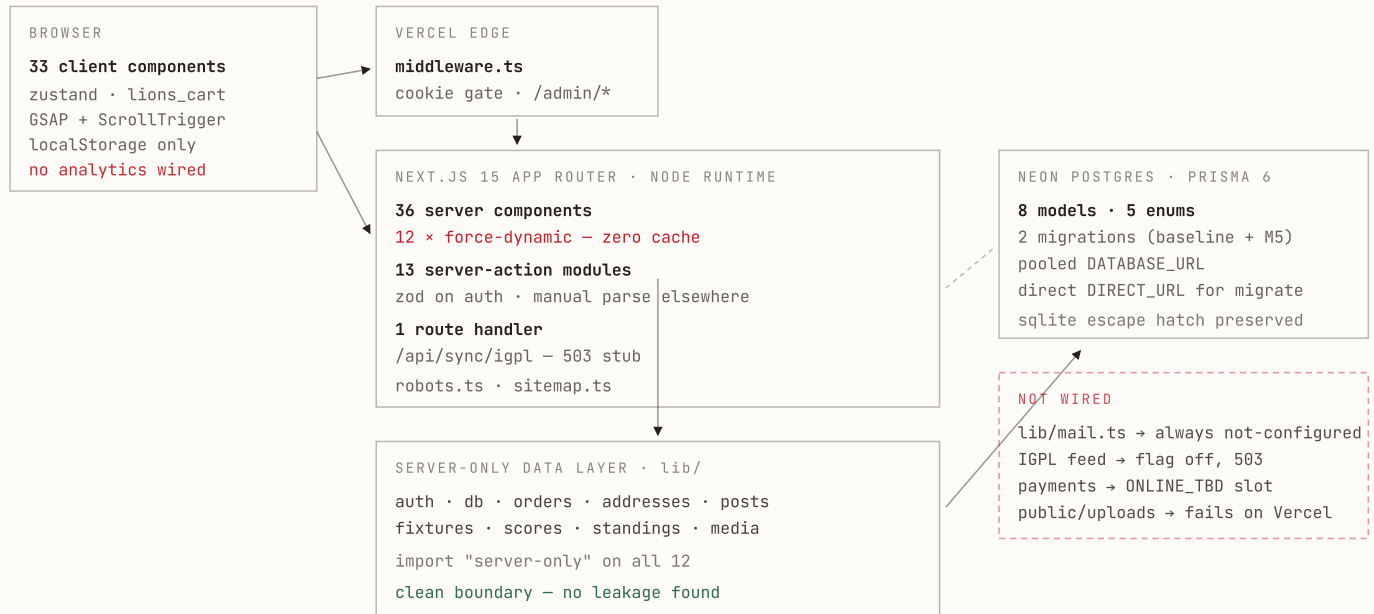
Four P0 items, described in full in sections D, E and H. In one line each: **orders can be placed but never fulfilled** (no admin order UI exists); **product image uploads will fail on Vercel** (writes to a read-only filesystem); **63 MB of images deploy on every push** including four 7 MB portraits nothing renders; and **every content page is force-dynamic**, so the site has no caching layer at all.

THE HEADLINE JUDGEMENT

Do not rewrite this application. The backend deserves to survive intact. The work is *subtractive on the frontend* — consolidating four stacked CSS systems into one — and *additive on the backend* — finishing the order loop that is already 80% built. A ground-up redesign would throw away the best code in the repository.

B Architecture

Traced from imports, not inferred from the folder names.



The shape is sound and conventional for Next.js 15: server components read through a `server-only` lib layer; mutations go through server actions that re-authorize; the client holds nothing but cart state. There is **no API layer for the frontend to over-fetch from**, because there almost isn't one — a single route handler, and it is a deliberate stub. That is a good decision, not a gap.

The dashed box is the honest part of the picture. Four capabilities are scaffolded with correct types and no implementation: email, the IGPL feed, payments, and image storage. Each is documented in-place as unfinished. That discipline is why this audit is short on surprises.

c Route inventory

All 39 routes. Rendering mode and data source verified per file.

PUBLIC – FRANCHISE & CONTENT

ROUTE	PURPOSE	DATA SOURCE	RENDER	KEY ISSUE	PRI
/	Franchise home, 7 chapters	Fixture + MediaCoverage + Product	dynamic	No per-page metadata; inherits layout title. No OG image.	P1
/the-club	Franchise story	Static, brochure-cited	static	Sound. Could be ISR-cached.	P3
/the-pride	City & mark	Static	static	Sound.	P3
/players	Roster, 4 players	data/players.ts	static	Roster is a TS file, not the DB – no admin can edit it.	P2
/golf-development	Academy thesis	Static	static	Sound.	P3
/vimtra-ventures	Owner profile	Static	static	Sound.	P3
/partners	Partners + 4 tiers	Static, brochure p.13/19	static	Partner logos are text only – no logo assets.	P2
/invest	Investment case	Static, brochure p.17–19	static	Sound. Financial figures all cited.	P3
/gallery	Image grid	Hardcoded in component	static	Not admin-manageable; no Gallery model.	P2
/contact	Enquiry form	–	static	Submissions are console-logged and lost.	P0
/privacy · /terms	Legal	Static	static	Sound.	P3

PUBLIC – SEASON DATA

ROUTE	PURPOSE	DATA SOURCE	RENDER	KEY ISSUE	PRI
/fixtures	2026 calendar	Fixture	dynamic	Correct. Cache candidate – fixtures change weekly, not per request.	P1
/scores	Round boards	Fixture + Score	dynamic	N+1 : one query per fixture. Position sort is fixed in JS, not SQL.	P1
/leaderboards	3 standings boards	Standing	dynamic	Single query, split in memory. Correct.	P2

ROUTE	PURPOSE	DATA SOURCE	RENDER	KEY ISSUE	PRI
/news	Editorial + press + social	Post + MediaCoverage	dynamic	3 parallel queries – good. Never cached.	P1
/news/[slug]	Article	Post	dynamic	Unsanitized bodyHtml via dangerouslySetInnerHTML. No generateStaticParams.	P1

COMMERCE

ROUTE	PURPOSE	DATA SOURCE	RENDER	KEY ISSUE	PRI
/shop	Catalog	Product (active only)	dynamic	Single query. No pagination, no filter, no sort at 21 items – fine now.	P2
/product/[id]	Detail	Product	dynamic	Has generateMetadata. No product JSON-LD.	P1
/cart	Cart	zustand / localStorage	client	Prices are client snapshots – correctly re-priced server-side at checkout.	P2
/checkout	Address + payment	Address book	dynamic	862-line client component. Payment is COD / offline only.	P2
/orders/[id]	Confirmation	Order, ownership-checked	dynamic	Correct – 404s on another user's id rather than redirecting.	–
/orders	–	–	–	Does not exist. A buyer cannot list their own orders.	P0

ACCOUNT & ADMIN

ROUTE	PURPOSE	AUTH	KEY ISSUE	PRI
/sign-in · /sign-up	Auth	public	No rate limiting on credential attempts.	P1
/profile	Account settings	requireUser	No order history, no address book UI despite both models existing.	P0
/admin	Dashboard, 7 stat tiles	requireAdmin	9 sequential-ish counts in one Promise.all – acceptable.	P3

ROUTE	PURPOSE	AUTH	KEY ISSUE	PRI
/admin/products + [id]/edit	Catalog CRUD	requireAdmin	Image upload writes to local disk — breaks on Vercel.	P0
/admin/inventory	Bulk stock	requireAdmin	Transactional, validated. Good.	—
/admin/fixtures + [id]/edit	Calendar CRUD	requireAdmin	Sound.	—
/admin/scores	Score rows	requireAdmin	Scores are free-text strings — no numeric validation.	P2
/admin/leaderboards	Standings	requireAdmin	Upsert on (season, board, rank). Sound.	—
/admin/news + [id]/edit	TipTap editor	requireAdmin	Stores raw HTML unsanitized.	P1
/admin/media + [id]/edit	Press coverage	requireAdmin	Sound.	—
/admin/users	Role management	requireAdmin	Self-lockout guards present. Good.	—
/admin/orders	—	—	Does not exist. Orders cannot be fulfilled, shipped or cancelled.	P0
/api/sync/igpl	Cron entry	CRON_SECRET	Auth is fail-open if the secret is unset; accepts secret in a query param.	P1

"Render" reflects the actual directive in the file. Twelve routes declare `export const dynamic = "force-dynamic"` ; the rest are statically rendered by default because they read no database.

D Frontend audit

The stylesheet is the single largest liability in the repository.

Four design systems are stacked in one 8,435-line stylesheet

P 0

`app/globals.css` contains **four separate `:root` token blocks** (lines 10, 1441, 1717, 2711), each declaring an overlapping palette under a different prefix: the legacy M0 system (`--ink` , `--crimson-600`), the "Grandstand" layer (`--gs-*` , 34 tokens), the "Shared Page Kit" (`--hp-*` , 12 tokens), and the "Art Direction System 2026" (`--cm-*`). Class prefixes tell the same story: **233 `.hp-` selectors**, **106 `.nv-`** , **105 `.tb-`** , **103 `.cm-`** , **87 `.hm-`** , plus `.vv-` , `.gd-` , `.pt-` , `.nw-` , `.plx-` , `.ss-` — a bespoke namespace per page.

Consequences that are already visible: **199 media queries** with no shared breakpoint scale, **44 `!important` declarations** fighting the cascade, and **four hero components** (`PageHero` , `PageMasthead` , `StoryHero` , `home/Hero`) that do the same job with different markup. Tailwind is configured with a proper token scale that most of the CSS ignores.

`app/globals.css` — 8,435 lines • 199 @media • 44 !important • 4 :root blocks

A 601 KB PNG logo loads on every page, unoptimized on one path

P 1

`/assets/logo-lion.png` is **601 KB** and appears in the header (with `priority`), the footer, the admin shell, both auth pages, as the favicon, and as `FALLBACK_LOGO` for every product without an image. In `components/Loader.tsx:64` it is a raw `<img>` with the Next lint rule disabled — so the splash screen pulls the full 601 KB with no resizing or format conversion, in front of the hero, on first visit. A 48 px logo should be roughly 8 KB.

`components/Loader.tsx:63-64` • `components/Nav.tsx:199` • `lib/products.ts:32`

Component duplication follows the CSS duplication

P 1

Three page-hero components with near-identical props (`eyebrow` , `title[]` , `lead / line` , `stats`) split arbitrarily across pages — `StoryHero` on 6 pages, `PageHero` on 8, `PageMasthead` on 4. Separately, `CheckoutFlow.tsx` is **862 lines in a single client component** and `InventoryTable.tsx` is 569; both mix form state, validation, layout and submission.

`components/site/{PageHero,PageMasthead,StoryHero}.tsx` •
`components/shop/CheckoutFlow.tsx`

GSAP + ScrollTrigger ships to seven routes

P 2

GSAP with ScrollTrigger is ~70 KB gzipped and is imported by `Hero` , `Sections` , `PlayerExperience` , `ScoreExperience` , `Chapters` , `FullBleedStatement` and `PageHero` . The motion work is genuinely good — every primitive no-ops under `prefers-reduced-motion` and content is never hidden by CSS alone, so a JS failure still renders the page. But most of these reveals are a fade and a 20px rise that a 30-line IntersectionObserver would produce. `components/Reveal.tsx` already is that observer, and it coexists with GSAP doing the same thing.

`components/motion/gsap.ts` + 7 consumers • `components/Reveal.tsx`

What is working on the frontend

- Hydration is handled correctly – `useCartHydrated()` gates every cart read so SSR and first paint agree.
- The splash loader shows once per session and is skipped entirely under reduced motion (`Loader.tsx:34-38`) – the right call.
- Only one raw `<img>` in the whole app; everything else uses `next/image` .
- The mobile nav overlay manages focus properly: focuses close on open, restores the trigger on close, traps Escape, and sets `tabIndex={-1}` on hidden items.

E Backend & data-flow audit

Traced database → lib → action → component → user for every commerce and content path.

The order pipeline has no fulfilment end

P8

`lib/orders.ts` exports a complete admin lifecycle API — `listOrdersForAdmin` , `getOrderByIdForAdmin` , `setOrderStatus` , `setPaymentStatus` , `cancelOrderAndRestock` , and a validated `FORWARD` state machine. **Every one of them has zero callers**. There is no `/admin/orders` route and no `/profile/orders` route.

The live consequence: a customer places an order, **real stock is decremented**, and nobody can see it, mark it paid, ship it, or cancel it. Because `cancelOrderAndRestock` is unreachable, **stock lost to abandoned or fraudulent orders never returns** — the catalog silently bleeds inventory.

`app/checkout/actions.ts:236-238` already calls `revalidatePath("/profile/orders")` and `revalidatePath("/admin/orders")` , both pointing at routes that do not exist. The comment at `app/orders/[id]/page.tsx:37` refers to "the separate `/admin/orders/[id]` route (Phase 5.5)" — Phase 5.5 was never built.

`lib/orders.ts:300-420` (no callers) · `app/checkout/actions.ts:236-238` · `app/admin/` (no orders dir)

Product image uploads write to a filesystem Vercel does not have

P8

`app/admin/products/actions.ts:32-35` does `mkdir(path.join(process.cwd(), "public", "uploads"))` then `writeFile` . On Vercel the filesystem is read-only apart from `/tmp` , and `/tmp` does not survive between invocations. Every admin image upload will fail in production — and because `public/uploads/` is git-ignored, the three files sitting there locally will not deploy either, so any product already pointing at `/uploads/...` renders a broken image.

The code knows this: there is a "DEPLOYMENT CAVEAT" block at line 62 saying exactly this. It was documented and then not fixed. Failures do surface as a clean form error rather than a crash, which is the right handling of the wrong situation.

`app/admin/products/actions.ts:26-46, 60-71` · `.gitignore:/public/uploads/`

Contact form submissions are written to stdout and discarded

P8

`submitContact` validates three fields, then `console.log` s the enquiry — including name, email, phone and city — and returns a success message to the visitor promising a reply "within two working days." Nothing is stored and nothing is sent. Every enquiry from the public contact page, the merchandise CTA and the partner CTAs is lost. Separately, writing customer PII to server logs is poor data handling regardless of the missing persistence.

`app/contact/actions.ts:22-34`

Nothing is cached — twelve routes are force-dynamic

P1

`/`, `/shop`, `/product/[id]`, `/news`, `/news/[slug]`, `/fixtures`, `/scores`, `/leaderboards` and four others all declare `force-dynamic`. There is no `revalidate`, no `unstable_cache`, no `generateStaticParams` anywhere in the app. Every visitor to the homepage triggers three Neon round-trips from a cold serverless function.

The irony is that the admin actions are meticulous about `revalidatePath` — 22 calls across 13 modules. That machinery is doing nothing, because there is no cache to invalidate. Switching these routes to `export const revalidate = 300` makes the existing `revalidatePath` calls start earning their keep and cuts TTFB to a CDN hit.

12 × force-dynamic · 22 × revalidatePath · 0 × revalidate

/scores runs an N+1 query

P1

`app/scores/page.tsx:71-74` maps over fixtures and awaits `listScoresForFixture(f.id)` for each. At four fixtures this is four extra round trips per request on a force-dynamic page; across a full 2026 calendar it scales linearly. The fix is one line: `prisma.fixture.findMany({ include: { scores: true } })`, or a single `score.findMany({ where: { fixtureId: { in: ids } } })` grouped in memory. The file's own comment acknowledges the pattern and accepts it at current scale.

`app/scores/page.tsx:70-75`

Validation is inconsistent across server actions

P2

`app/(auth)/actions.ts` uses zod schemas properly. Every other action — checkout, products, fixtures, scores, standings, news, media — hand-rolls `String(formData.get(x) ?? "").trim()` helpers. The hand-rolled parsing is defensive and largely correct (clamping, rounding, enum allow-lists), but it is duplicated in nine files with slightly different shapes, and zod is already a dependency. Notable gaps: score values accept any string; product `price` has no upper bound; `sortOrder` is `Number(...) || 0` which silently swallows garbage.

`app/(auth)/actions.ts` vs 9 × manual parse helpers

Receipt email passes a hardcoded zero

P2

`app/checkout/actions.ts:227` calls `sendOrderReceipt({ totalRupees: 0, ... })` with an inline comment saying the real value lands in Phase 5.5. Harmless today because `lib/mail.ts` always returns `not-configured` — but it becomes a wrong-number customer email the moment SMTP is wired.

`app/checkout/actions.ts:224-230` · `lib/mail.ts:47-62`

F Database audit

8 models, 5 enums, 2 Postgres migrations. Reviewed against the queries that use them.

MODEL	INDEXES	ASSESSMENT
User	unique email	Correct. Cascade to Session and Address; <code>Restrict</code> on Order via <code>userId</code> — a user with orders cannot be hard-deleted, which is right.
Session	unique token, <code>userId</code>	Correct. No cleanup job for expired rows — the table grows without bound at 30-day TTL.
Product	pk only	Missing index on <code>active</code> . <code>listProducts()</code> filters <code>where: { active: true }</code> on every public page. Fine at 21 rows, wrong by default. <code>sku</code> is not unique.
Fixture	unique slug	No index on <code>(sortOrder, dateStart)</code> , which is the only ordering ever used. No index on <code>status</code> , which every page filters on.
Score	<code>fixtureId</code>	Indexed correctly for the N+1 query it feeds. All score columns are <code>String?</code> — <code>r1 ... r4</code> , <code>total</code> , <code>position</code> . This is why <code>/scores</code> has to re-sort positions in JavaScript.
Standing	unique (season, board, rank)	Good — the unique key is exactly the upsert key. <code>extra</code> is a JSON-encoded <code>String</code> rather than <code>Json</code> , a carryover from the SQLite baseline that Postgres no longer needs.
Post	(status, publishedAt), slug	Well indexed for both the public and admin query. Correct.
MediaCoverage	(active, kind, sortOrder)	Composite index matches the query exactly. Best-indexed model in the schema.
Address	<code>userId</code>	Correct. <code>isDefault</code> uniqueness is enforced in transactions rather than by constraint — acceptable, though a partial unique index would be stronger.
Order	(<code>userId</code> , <code>createdAt</code>), (<code>status</code> , <code>paymentStatus</code>), <code>createdAt</code>	Indexed for admin lists that <i>do not exist yet</i> . Ready for the P0 fix.
OrderItem	<code>orderId</code> , <code>productId</code>	Correct. <code>onDelete: Restrict</code> on Product preserves order history.

Where the frontend expects data the schema can't reliably give

- Roster.** `/players` and `/scores` both read `data/players.ts` — a TypeScript file. There is no `Player` model. An admin cannot add or edit a player, and `/scores` marks "our" rows by string-matching player names against that file (`ROSTER_NAMES`), which breaks on any spelling variance in an IGPL feed.

- **Gallery.** Images are hardcoded in `GalleryGrid.tsx`. No model, no admin surface.
- **Partners.** Hardcoded in the page. No model — adding a sponsor requires a deploy, which is a poor fit for a commercial surface.
- **Contact messages.** `CLAUDE.md` names a future `ContactMessage` model. It does not exist, which is why section E's P0 exists.

MIGRATION QUALITY

The Postgres path is one consolidated baseline plus one M5 migration; five historical SQLite migrations are preserved separately under `migrations_sqlite/` with their own lock file. This dual-schema arrangement is documented clearly and the seed script runs against either. It is unusual but deliberate, and it works — **keep it, don't refactor it.**

G Performance audit

Measured from the actual asset tree and rendering directives, not a Lighthouse guess.

63 MB of images deploy on every push; 27 MB of it renders nowhere

P0

`public/` is **63 MB**. The four player portraits are **6.7–7.2 MB each** — 27.5 MB combined. Nothing renders them: `lib/image-src.ts` maps each to a `-web.jpg` derivative of ~160 KB at render time. The originals exist only to be copied into every deployment.

The same pattern repeats across the tree: `subash-yammada.png` (1.9 MB → 107 KB derivative), `hero-golfer.png` (1.2 MB), `fan-thumb.png` (946 KB), nine product PNGs at 500–780 KB each. **Fifteen files over 500 KB have a derivative that is the only thing ever served.** Moving the originals out of `public/` into a `design-source/` directory outside the deploy — or into `git-lfs` — reclaims roughly 40 MB with zero rendering change.

`du -sh public` → 63M · 37 × `-web` derivatives in `public/` alongside their originals

Core Web Vitals — expected behaviour and cause

Metric	Risk	Cause traced in this codebase	Concrete fix
TTFB	HIGH	12 routes <code>force-dynamic</code> ; every request is a cold serverless invocation plus 1–5 Neon queries. No cache layer exists.	<code>export const revalidate = 300</code> on the eight content routes; keep <code>force-dynamic</code> only on <code>/checkout</code> , <code>/orders/[id]</code> , <code>/profile</code> , <code>/admin/*</code> .
LCP	HIGH	The splash loader paints a 601 KB unoptimized <code></code> in front of the hero on first visit; the hero image itself then loads behind it.	Export a 96 px <code>logo-lion-web.png</code> (~8 KB) and switch <code>Loader.tsx</code> to <code>next/image</code> . Add <code>priority</code> + explicit <code>sizes</code> to the hero figure.
CLS	LOW	<code>next/font</code> with <code>display: swap</code> and CSS variables — no FOUT layout shift. All images use <code>next/image</code> with <code>dimensions</code> or <code>fill</code> .	None needed. This is already right.
INP	MEDIUM	GSAP + <code>ScrollTrigger</code> on 7 routes; <code>Nav</code> attaches a scroll listener (correctly passive) that calls <code>setState</code> on every crossing of 40 px.	Replace the 7 simple reveals with the existing <code>Reveal.tsx</code> observer; keep GSAP only for the home hero timeline. Debounce or use an <code>IntersectionObserver</code> sentinel for the nav scroll state.

Bundle

- 33 client components against 36 server components — a reasonable ratio, but `components/home/Sections.tsx` (562 lines) is entirely client-side because of GSAP, when only the reveal wrappers need to be.

- **TipTap** (`@tiptap/core` , `react` , `starter-kit`) is a dependency used by exactly one admin route. Confirm it is not entering the public bundle — `PostEditor.tsx` is client-side and admin-only, so route-level code splitting should isolate it. Worth verifying with `@next/bundle-analyzer` .
- Correct call already made: `/news/[slug]` renders pre-stored `bodyHtml` rather than loading the editor to render JSON.
- **No third-party scripts at all** — no analytics, no tag manager, no chat widget. Excellent for performance; a gap for the business (see section J).

H Security audit

No critical vulnerability found. Four issues worth fixing before launch.

Stored HTML is rendered without sanitization

P1

`app/news/[slug]/page.tsx:113` renders `post.bodyHtml` through `dangerouslySetInnerHTML`. That value arrives as a plain form field in `updatePostAction` (`app/admin/news/actions.ts:85`) and is stored verbatim. There is **no sanitizer anywhere in the project** — no `DOMPurify`, no `sanitize-html`.

Severity is bounded by the ADMIN precondition and Next.js's built-in Server Action origin check, so this is not remotely exploitable today. It becomes serious the moment a non-admin editor role is introduced, or if any admin account is compromised — at which point it is persistent script execution on a public page. Fix: sanitize on write in `updatePostAction` with an allow-list (`isomorphic-dompurify`), not on read.

The other two `dangerouslySetInnerHTML` sites are safe and correctly documented: `ToastHost` and `PlayerExperience` both render in-repo copy with no user input path.

`app/news/[slug]/page.tsx:113` · `app/admin/news/actions.ts:85` · no sanitizer
in `package.json`

The cron endpoint fails open and accepts its secret in a query string

P1

`app/api/sync/igpl/route.ts:27` reads `if (secret && auth !== ...)`. If `CRON_SECRET` is unset or empty in the environment, **the authorization check is skipped entirely** and the endpoint is public. Today that only exposes row counts behind a 503, but it is the wrong default and the danger grows when the sync is implemented. It also accepts the secret as `?key=`, which puts it into access logs, browser history and any Referer header.

Confirmed: the local `.env` contains only `ADMIN_EMAIL`, `ADMIN_PASSWORD`, `DATABASE_URL` and `DIRECT_URL` — no `CRON_SECRET`, so this endpoint is unauthenticated in the current environment. Fix: fail closed when the secret is absent, and drop the query-param path.

`app/api/sync/igpl/route.ts:24-29` · `.env` (4 keys present)

No rate limiting on authentication

P1

`signIn` can be called without limit. `verifyCredentials` does defend against user enumeration by comparing against a dummy bcrypt hash when the email is unknown (`lib/auth.ts:64-68`) — a thoughtful detail — but there is no attempt counter, lockout, or IP throttle. bcrypt at 10 rounds is the only brake, and it is also a self-inflicted DoS vector: concurrent sign-in attempts each burn ~100 ms of serverless CPU.

`lib/auth.ts:56-70` · `app/(auth)/actions.ts:35-53`

Upload type checking trusts the client-declared MIME type

P2

`app/admin/products/actions.ts:27` validates `upload.type` — a value the browser supplies and an attacker controls. The extension is then derived from that same value. Mitigated in practice by three things: the route is admin-only, the file is renamed to a UUID, and `X-Content-Type-Options: nosniff` is set globally in `next.config.mjs`. Defense-in-depth fix: check magic bytes, and move to blob storage on a separate origin — which the P0 in section E requires anyway.

`app/admin/products/actions.ts:26-31` · `next.config.mjs:12`

Verified correct

- `.env` is **not tracked in git** (confirmed with `git ls-files`) and `.gitignore` covers env files, SQLite databases, backups and `public/uploads`.
- No secret is exposed to the client — the only `NEXT_PUBLIC_` variable is the site URL.
- Prisma parameterizes everything; there is no raw SQL and no `$queryRawUnsafe` in the repository.
- Open-redirect is handled: `safeNext()` rejects anything not starting with a single `/`.
- Order ownership is checked in the data layer, and `/orders/[id]` returns 404 rather than a redirect for another user's id — correctly avoiding an existence oracle.
- Security headers are applied to every route, with HSTS correctly gated to production only.
- Session cookie is `httpOnly`, `sameSite: lax`, `secure` in production.

THE CSP GAP, AND WHY IT IS DEFERRED

`next.config.mjs` sets six security headers but no Content-Security-Policy, with a comment explaining that a nonce-based CSP is scheduled alongside the design-system consolidation because the current code relies on inline styles. That reasoning is correct and the sequencing is right — **CSP lands in Phase 2, after the CSS refactor, not before.**

I Accessibility audit

Better than typical. Three real gaps.

Already correct

- Skip link to `#main-content`, with `tabIndex={-1}` on `<main>` so focus actually moves — the detail most implementations miss (`app/layout.tsx:61-68`).
- Exactly one `<h1>` per page, emitted by the shared hero components; sections use `<h2>` with `aria-labelledby`.
- Alt text is genuinely descriptive where it matters ("A links fairway at twilight, a lone figure far down the hole") and correctly empty on decorative marks.
- A global `prefers-reduced-motion` block at `globals.css:1517` plus a JS-level `reduced()` guard in every GSAP primitive — belt and braces.
- Mobile nav is a proper dialog: `role="dialog"`, `aria-modal`, Escape handling, focus restoration, and `tabIndex={-1}` on off-screen links.
- A documented two-ring focus pattern that reads on light, gold and charcoal surfaces (`globals.css:1613`).

The desktop mega-menu opens on hover and is not keyboard-dismissible in place

P1

`Nav.tsx:219-227` opens the panel on `onMouseEnter` and `onFocus`. A keyboard user tabbing across the three triggers opens all three in sequence with no way to move through the panel contents in reading order, and `onMouseLeave` on the header closes it out from under a pointer travelling diagonally. `aria-haspopup="true"` should be `"menu"` or the pattern should move to a disclosure with explicit click/Enter toggling.

components/Nav.tsx:212-235, 284-291

Score and standings tables need scope and caption semantics

P1

The `/scores` and `/leaderboards` boards are the most data-dense surfaces on the site and the ones a screen-reader user most needs structure from. Verify each table emits `<caption>`, `scope="col"` / `scope="row"`, and that the `is-lions` row marker is conveyed by more than colour.

components/season/ScoreExperience.tsx • components/season/StandingsBoard.tsx

Contrast tokens need auditing after the CSS consolidation

P2

`--muted: #6B635C` on `--cream-100: #F4F0E8` is roughly 4.9:1 — passes AA for body text, fails AA for the 10-12 px uppercase labels it is used on throughout (`.hp-eyebrow`, `.nv-mega-d`, stat labels). Small text needs 4.5:1 but these sit near the boundary at sizes where legibility is already marginal. Resolve this once, in the consolidated token set, rather than per page.

`globals.css :root` blocks • `--muted` used at 10-12px across `.hp-/.nv-/.cm-` families

J SEO audit

Foundations are in place. Sports-specific opportunity is entirely untapped.

NEXT_PUBLIC_SITE_URL is unset — the sitemap will publish localhost

P0

`app/robots.ts:6` falls back to `http://localhost:3000` when the variable is missing, and the local `.env` does not contain it. Every URL in `sitemap.xml`, plus the `host` and `sitemap` directives in `robots.txt`, would point at localhost. **Verify this is set in the Vercel environment before launch** — it is a one-variable fix that silently destroys indexing if missed.

`app/robots.ts:6-8` · `.env` contains 4 keys, none of them this one

No Open Graph, no Twitter cards, no OG image anywhere

P1

37 files export `metadata` with title and description, which is good coverage. None export `openGraph` or `twitter`. For a sports franchise — whose primary distribution is a link shared into WhatsApp, X and Instagram — every share currently renders as a bare grey link. Add `metadataBase` plus a default `openGraph` block in `app/layout.tsx`, and an `opengraph-image.tsx` per dynamic route (`/news/[slug]`, `/product/[id]`) using Next's built-in image generation.

`37 × metadata exports` · `0 × openGraph` · `0 × twitter`

The home page has no metadata of its own

P1

`app/page.tsx` exports no `metadata`, so the homepage inherits the root layout's generic title. Every other public page has a specific one. The single most-linked URL on the site is the one without a tailored title and description.

`app/page.tsx` (no metadata export) · `app/layout.tsx:38`

Zero structured data — the biggest untapped SEO asset here

P1

No JSON-LD anywhere. A sports organisation site has unusually rich schema.org coverage available, and this one already holds the underlying rows:

- `SportsOrganization` on `/the-club` — name, logo, sport, parent organisation, sameAs links.
- `SportsEvent` per `Fixture` row on `/fixtures` — this is what surfaces event rich results.
- `Person` + `Athlete` per roster entry on `/players`.
- `NewsArticle` on `/news/[slug]` — `headline`, `datePublished`, `author` and `image` all already exist on the `Post` model.
- `Product` + `Offer` on `/product/[id]` — price, availability from `stock`, currency INR. Enables shopping rich results.
- `BreadcrumbList` site-wide.

`grep for ld+json / JSON-LD across app + components` → 0 matches

Correct already

- `sitemap.ts` enumerates products and published posts from the live database, with try/catch fallbacks so a cold DB doesn't fail the build. Drafts and archived posts are correctly excluded.
- `robots.ts` disallows `/admin` , `/api/` , `/profile` , `/cart` , `/checkout` , `/orders` , and both auth routes.
- Private routes additionally set `robots: { index: false }` in their own metadata — correct belt-and-braces.
- URL structure is clean, flat and human-readable throughout.

K 2026 design direction

Consolidation, not reinvention. The visual language is already here — it is buried under three older ones.

The honest finding is that a redesign is not what this site needs. Reading the four CSS eras in order, the newest layer — the "Art Direction System 2026" / Club Module (`.cm-` , `globals.css:2697+`) — is already a coherent, editorial, photography-led language: full-bleed story heroes, a 4/8/12 editorial track, numbered chapters, an ink/ivory surface alternation, generous display type. It reads like a modern sports organisation. The problem is that it covers roughly a third of the site while `.hp-` , `.tb-` and the original M0 system cover the rest.

THE DIRECTION

Finish the Club Module across all 21 public pages and delete the other three systems. That is a subtractive project with a known endpoint, not a speculative redesign. It resolves the visual inconsistency, the CSS weight, the component duplication, and unblocks the CSP — one piece of work, four problems.

What the language should carry, and where it already does

QUALITY	CURRENT STATE	ACTION
Chennai identity	Strong in copy, weak in image. Photography is generic stock golf — captions deliberately avoid naming venues because the shots aren't ours.	COMMISSION Real Chennai and match-day photography. This is the single highest-leverage non-code investment, and no amount of CSS substitutes for it.
Competition & match-day	Honest but inert. <code>/scores</code> shows unlit boards; <code>/fixtures</code> is a calendar with no live state.	BUILD A "next fixture" countdown strip and a LIVE state — the <code>FixtureStatus.LIVE</code> enum already exists and nothing renders it distinctly.
Team identity	Excellent. Crimson/gold/ink is distinctive and consistently applied where the Club Module reaches.	KEEP Do not change the palette. Consolidate the four token sets that express it.
Professionalism & trust	Very strong — the honest empty states are more trustworthy than fabricated numbers would be.	KEEP Protect this. It is the project's best decision.
Sponsorship visibility	Weak. Partners appear as styled text; no logo lockups, no home-page partner band.	ADD A partner model + logo assets, and a restrained partner strip above the footer.
Community	Absent. No fan-facing surface beyond the shop.	LATER Out of scope until the P0s are closed.

What to avoid

The brief warns against generic AI-website aesthetics, and this codebase has largely avoided them — but two habits are creeping in and should be pruned during consolidation: **numbered section markers used decoratively** (`01/02/03` is meaningful on `/invest` 's tiers and `/the-club` 's build sequence, arbitrary on `/news` where three categories are not a sequence), and **eyebrow labels above every single section**, which stop functioning as signposts once they are universal.

L Page-by-page plan

Recommended structure for every major surface. Existing sections in order; changes marked.

Home — /

The current seven-chapter narrative (Hero → Statement → Club → Season → Development → Media → Shop → Closing) is well-conceived and reads as one editorial piece. It already pulls real fixtures, real press coverage, and a real product count from the database. **Do not reorder it.** Four changes:

- **P0** Add `metadata` with `openGraph` — the most-shared URL on the site currently has none.
- **P1** Add a **next-fixture strip** directly under the hero with a countdown and a LIVE state. The data is already loaded in `page.tsx:108-115`; the hero renders it as static text.
- **P1** Add a **partner band** above the closing chapter — currently the commercial story is invisible on the homepage.
- **P1** Switch to `revalidate = 300`. There is no reason this page recomputes per visitor.

Scores & Leaderboards — /scores · /leaderboards

The empty-state discipline here is the best thing on the site and must survive any redesign. **P1** Fix the N+1. **P1** Add table semantics (section I). **P2** Consider typing the `Score` numeric columns as `Int?` so ordering can happen in SQL rather than a JS re-sort. **P2** Render `FixtureStatus.LIVE` distinctly — a live event should not look like an upcoming one.

Shop, Product, Cart, Checkout

The buying half is complete and correct. The owning half is missing. **P0** Build `/profile/orders` and `/orders` so a buyer can see what they bought. **P1** Add `Product` JSON-LD with `availability` derived from `stock`. **P2** Split `CheckoutFlow.tsx` (862 lines) into address / payment / summary steps. **P2** Surface stock scarcity ("3 left") — the field exists and is unused on the buyer side beyond hiding the button at zero.

News — /news · /news/[slug]

The three-way separation of franchise editorial, press coverage and social is a genuinely good information-architecture decision — a reader always knows who is speaking. Keep it. **P1** Sanitize `bodyHtml`. **P1** Add `NewsArticle` JSON-LD and per-article OG images. **P1** Add `generateStaticParams` + ISR — articles are the most cacheable content on the site and are currently the least cached.

Contact — /contact

P0 Add a `ContactMessage` model, persist submissions, and build `/admin/messages`. Until then the page tells visitors they will hear back and guarantees they will not. **P1** Stop logging PII to stdout.

Players — /players

P2 Promote the roster from `data/players.ts` to a `Player` model with an admin surface, and key `/scores` row-marking off a player id rather than a lowercased name string. **P1** Move the four 7 MB originals out of `public/`. **P2** Add `Athlete` JSON-LD.

Gallery & Partners

P2 Both are hardcoded arrays in components. Both are surfaces the franchise will want to change without a deploy — particularly Partners, which is commercial. Give each a model and an admin page, following the `MediaCoverage` pattern already in the codebase, which is the cleanest CRUD example to copy.

The Club · The Pride · Golf Development · Vimtra Ventures · Invest

P3 These are in the best shape on the site — Club Module styling, fully brochure-cited, honest. They need nothing beyond ISR caching and inclusion in the CSS consolidation. **Leave them alone.**

M

Design system

One token set to replace four. Values taken from the existing tokens, not invented.

METHOD

Promote the `--gs-*` Grandstand tokens — already the most complete set, with 34 tokens covering colour, space, radius, elevation, duration and easing — to be the single source, extend them with what `.cm-` needs, then delete `--hp-*` and the two legacy `:root` blocks. `tailwind.config.ts` already declares most of this scale and is currently under-used; make it the authority and let the stylesheet reference it.

TYPE SCALE

ROLE	FACE	SIZE	WEIGHT / TRACKING	USE
Display	Sora	clamp(40px, 7vw, 84px)	800 / -0.035em	Hero titles only. One per page.
H1	Sora	clamp(32px, 4.6vw, 54px)	800 / -0.03em	Page mastheads.
H2	Sora	clamp(24px, 3vw, 34px)	700 / -0.025em	Chapter titles.
H3	Sora	19px	700 / -0.015em	Card and row titles.
Lead	Fraunces	clamp(18px, 2vw, 22px)	400 / normal	Opening paragraph. Already loaded, barely used.
Body	Manrope	16px / 1.68	400	Running text. Cap at 68ch.
Small	Manrope	14px / 1.6	500	Captions, meta.
Label	Manrope	11px	700 / 0.22em caps	Eyebrows. Must clear 4.5:1 — see section I.
Data	Sora	tabular-nums	700	Scores, prices, ranks, countdowns.

SPACING — ONE 4PX-BASED SCALE, REPLACING 199 AD-HOC MEDIA QUERIES

`4 · 8 · 12 · 16 · 24 · 32 · 48 · 64 · 96 · 128 · 160`, exposed as `--gs-space-*` (`xs` → `3xl`, already defined). Four breakpoints only: `640 · 768 · 1024 · 1280`. Section rhythm is a floor, not a fixed height — the Club Module already works this way and it is the right model.

COMPONENT INVENTORY TO BUILD ONCE AND REUSE

COMPONENT	REPLACES	NOTES
<PageHero>	PageHero + PageMasthead + StoryHero	One component, a <code>variant</code> prop: <code>story</code> / <code>masthead</code> / <code>compact</code> . Emits the single <code><h1></code> .

COMPONENT	REPLACES	NOTES
<Section>	.hp-sec, .cm-sec, .hm-sec	Surface (ivory / ink / paper / image) + size. Already exists; extend it.
<FixtureCard>	ad-hoc fixture markup ×3	Must render UPCOMING / LIVE / COMPLETED distinctly.
<ScoreBoard>	ScoreExperience internals	Semantic <table> with caption and scope. Lions row marked non-visually too.
<PlayerCard>	PlayerExperience internals	
<StoryCard>	news + media + gallery card variants	One card, three content shapes.
<PartnerLockup>	—	New. Needs logo assets first.
<DataTable>	.tb-* family (105 selectors)	Scroll container, sticky header, tabular figures.
<EmptyState>	.hp-empty + unlit board	Already excellent. Promote it, don't redesign it.
<Field>	.field overrides + 44 !important	Absorbs the Tailwind preflight workaround documented in CLAUDE.md.
<Button>	.cta-gold, .btn-*, .press	Three tiers, one shape system.

MOTION

Keep GSAP for exactly one thing: the home hero's orchestrated load timeline, which is a real composed moment. Everything else — the scroll reveals on seven routes — moves to the existing `Reveal.tsx` `IntersectionObserver`. Durations from the token set (fast 150 · base 300 · slow 700 · hero 1200), easing from `--gs-ease` . The existing rule that content is never hidden by CSS alone is a good rule; carry it forward.

N Priority matrix

Effort is engineer-days for one developer familiar with the codebase.

AREA	CURRENT	PROBLEM	RECOMMENDATION	PRI	EFFORT
Orders	BROKEN	No admin or buyer order UI; stock leaks permanently	Build <code>/admin/orders</code> + <code>/profile/orders</code> against the existing lib API	P0	3-4 d
Uploads	BROKEN	Writes to a read-only Vercel filesystem	Vercel Blob or S3; keep the same form contract	P0	1 d
Contact	BROKEN	Enquiries logged and lost; PII in stdout	<code>ContactMessage</code> model + <code>/admin/messages</code>	P0	1 d
Assets	POOR	63 MB deployed; 27 MB of it never rendered	Move originals out of <code>public/</code> ; resize the 601 KB logo	P0	0.5 d
SEO env	RISK	<code>NEXT_PUBLIC_SITE_URL</code> unset → localhost sitemap	Set in Vercel; assert at build time	P0	0.25 d
Caching	POOR	12 routes force-dynamic; 22 <code>revalidatePath</code> calls doing nothing	ISR on 8 content routes	P1	1 d
CSS system	POOR	8,435 lines, 4 token sets, 44 <code>!important</code>	Consolidate on Grandstand + Club Module; delete two eras	P1	8-12 d
XSS	RISK	Unsanitized <code>bodyHtml</code> rendered publicly	Sanitize on write with an allow-list	P1	0.5 d
Cron auth	RISK	Fails open without <code>CRON_SECRET</code> ; secret in query string	Fail closed; header only	P1	0.25 d
Auth abuse	RISK	No rate limit on sign-in	Upstash or in-memory limiter on <code>signIn</code>	P1	0.5 d
Social meta	MISSING	No OG/Twitter tags; shares render bare	<code>metadataBase</code> + OG defaults + generated images	P1	1.5 d
Structured data	MISSING	No JSON-LD; no event or product rich results	6 schema types from existing rows	P1	2 d
N+1 on /scores	POOR	One query per fixture	<code>include: { scores: true }</code>	P1	0.25 d

AREA	CURRENT	PROBLEM	RECOMMENDATION	PRI	EFFORT
Nav a11y	POOR	Hover-only mega menu; keyboard traversal broken	Disclosure pattern with explicit toggle	P1	1 d
Table a11y	POOR	Score boards lack caption/scope; state is colour-only	Semantic table primitive	P1	1 d
Docs	STALE	CLAUDE.md is ~2 milestones behind the code	Rewrite the status and layout sections	P1	0.5 d
Validation	MIXED	zod in auth only; 9 hand-rolled parsers	Shared zod schemas per action module	P2	2 d
Components	DUPLICATED	3 hero components; 862-line checkout	Unify per section M	P2	3 d
DB indexes	THIN	No index on Product.active , Fixture.status /sort	One migration	P2	0.5 d
Session GC	MISSING	Expired sessions accumulate forever	Cron delete on expiresAt < now()	P2	0.25 d
Content models	MISSING	Players, gallery, partners are hardcoded	Models + admin, following MediaCoverage	P2	4 d
GSAP weight	HEAVY	~70 KB on 7 routes for fade-and-rise	Keep for the hero; Reveal.tsx elsewhere	P2	1.5 d
CSP	DEFERRED	No CSP; blocked on inline styles	Nonce-based, after the CSS work	P2	1.5 d
Analytics	MISSING	No measurement of anything	Vercel Analytics + Speed Insights	P2	0.5 d
Score typing	WEAK	All score columns are strings	Int? + SQL ordering	P3	1 d
Payments	SCAFFOLD	COD / offline only; ONLINE_TBD unfilled	Razorpay into the existing enum slot	P3	3 d
Tests	NONE	One ad-hoc checkout script, no framework	Vitest on orders-totals , placeOrder , auth	P3	3 d

0 Implementation roadmap

Ordered so each phase is independently shippable and testable.

PHASE 1	Foundation & critical fixes Close all five P0s: order management UI, blob storage for uploads, <code>ContactMessage</code> persistence, asset diet, and <code>NEXT_PUBLIC_SITE_URL</code> . Update <code>CLAUDE.md</code> to match reality. No visual changes in this phase – it is the one that makes the site safe to sell from.	~6 days
PHASE 2	Design system consolidation Promote the Grandstand tokens to sole authority, extend with Club Module needs, delete <code>--hp-*</code> and both legacy <code>:root</code> blocks. Unify the three hero components. Build the component inventory in section M. Target: 8,435 → under 2,500 lines , zero <code>!important</code> . Land the nonce-based CSP at the end of this phase.	~10 days
PHASE 3	Core pages Migrate the home page, <code>/fixtures</code> , <code>/scores</code> , <code>/leaderboards</code> and <code>/shop</code> onto the consolidated system. Add the next-fixture strip, the LIVE state, and the partner band.	~6 days
PHASE 4	Data & backend ISR across the eight content routes. Fix the <code>/scores</code> N+1. Shared zod schemas. DB index migration. Session cleanup cron. Sanitize <code>bodyHtml</code> . Fail-close the cron auth. Rate-limit sign-in.	~5 days
PHASE 5	Secondary pages & content models Migrate the remaining public pages. Promote players, gallery and partners from hardcoded arrays to models with admin surfaces, following the <code>MediaCoverage</code> pattern.	~7 days
PHASE 6	Performance GSAP reduction. Bundle analysis. Image audit pass two. Verify LCP, CLS, INP and TTFB against real field data rather than lab numbers.	~3 days
PHASE 7	Accessibility & SEO Nav disclosure pattern. Table semantics. Contrast pass on the consolidated tokens. OG defaults and generated images. Six JSON-LD types. Full keyboard and screen-reader pass.	~5 days
PHASE 8	QA Vitest on the order and totals paths. Manual regression on the full commerce flow. Responsive testing at 375 / 768 / 1024 / 1440. Admin CRUD regression across all seven managers.	~4 days

PHASE 9

Production

~2 days

Environment audit against `.env.example`. `prisma migrate deploy` on the production database. Seed the admin and rotate the password immediately. Analytics and Speed Insights. Domain and DNS cutover. Post-deploy smoke test.

Roughly 48 engineer-days end to end. Phase 1 alone is what stands between the current state and a site that can safely take money.

P

Risk register

Changes recommended above that can break working behaviour, and how to contain each.

CHANGE	WHAT COULD BREAK	CONTAINMENT
Deleting CSS eras	Highest risk in the whole plan. 233 <code>.hp-</code> and 105 <code>.tb-</code> selectors are referenced from JSX across 21 pages. Deleting a rule that one page still depends on produces silent visual breakage that typechecks and lints clean.	Migrate page by page, never system by system. Delete a prefix only when <code>grep -rn "\.hp-" app components</code> returns nothing. Screenshot each page before and after at four widths.
Switching to ISR	Admin edits appear to "not save" if a <code>revalidatePath</code> target is wrong. Two already point at non-existent routes.	Audit all 22 <code>revalidatePath</code> calls against the real route tree in the same PR. Start with <code>revalidate = 60</code> , raise it after confirming invalidation works.
Moving image originals	Any database row or JSX path still pointing at an original renders 404. <code>lib/image-src.ts</code> maps known originals, but an admin-entered path bypasses the map.	Query <code>Product.img</code> , <code>Product.images</code> , <code>Post.coverImage</code> and <code>MediaCoverage.coverImage</code> for original filenames before deleting anything. Keep the <code>DERIVATIVES</code> map as the safety net.
Blob storage migration	The three files in <code>public/uploads/</code> are git-ignored and exist only on the developer's machine. They are already effectively lost on deploy.	Upload those three to blob storage and rewrite the affected <code>Product</code> rows <i>before</i> switching the write path.
Sanitizing <code>bodyHtml</code>	An over-strict allow-list silently strips formatting from existing published articles.	Sanitize on write only; leave stored rows untouched. Diff sanitized against original for every existing post before enabling.
Unifying the hero components	All three emit the page's only <code><h1></code> . A merge bug produces zero or two per page — invisible in review, damaging to SEO.	Assert exactly one <code><h1></code> per route in a smoke test.
Reducing GSAP	GSAP sets the "from" state in JS deliberately, so content stays visible if JS fails. A CSS-based replacement can hide content permanently on failure.	Preserve the invariant: the observer adds a visible class; the default state is visible, never hidden.
Adding a CSP	Inline <code>style</code> attributes are used throughout (e.g. <code>ProfileClient.tsx:40</code> , <code>invest/page.tsx</code> badge styles). A CSP without <code>'unsafe-inline'</code> breaks them silently.	Ship in report-only mode for a week first. This is exactly why the existing code defers CSP until after the CSS work — respect that sequencing.
Enabling the IGPL sync	Writing scraped data into <code>Fixture</code> / <code>Score</code> / <code>Standing</code> could overwrite hand-verified admin rows with unverified scraped ones — a direct violation of the project's own data-integrity rule.	Honour the fail-closed design already specified in <code>CLAUDE.md §6</code> . And per <code>CLAUDE.md §5</code> : get IGPL's agreement before wiring production to their undocumented internal endpoints.

q Verification checklist

Run before each phase merges. The first two already pass on `main` today.

<code>npx tsc --noEmit</code>	TypeScript strict. Currently passes.
<code>npm run lint</code>	ESLint / next-core-web-vitals. Currently clean. Note: <code>next lint</code> is deprecated for Next 16 — migrate to the ESLint CLI.
<code>npm run build</code>	Must run against a reachable database — <code>sitemap.ts</code> and every force-dynamic page query at build.
<code>npx prisma migrate deploy</code>	Applies the baseline + M5. Verify against a staging Postgres before production.
<code>npx prisma validate</code>	Run for <i>both</i> schemas — the SQLite escape hatch must not drift from the Postgres one.
<code>npm run db:seed</code>	Refuses without <code>ADMIN_EMAIL</code> + a 12-char <code>ADMIN_PASSWORD</code> . Rotate immediately after first login.
<code>npx @next/bundle-analyzer</code>	Confirm TipTap and GSAP are not in the public route bundles.
<code>du -sh public</code>	Gate: under 20 MB after the Phase 1 asset diet. Currently 63 MB.
<code>grep -c '!important' app/globals.css</code>	Gate: 0 after Phase 2. Currently 44.
<code>grep -c '^:root' app/globals.css</code>	Gate: 1 after Phase 2. Currently 4.
<code>curl -I \$SITE/api/sync/igpl</code>	Must return 401 without the bearer token — not 503. Verifies the fail-closed fix.
<code>curl \$SITE/robots.txt</code>	Sitemap and host must be the production origin, never localhost.
<code>curl \$SITE/sitemap.xml</code>	Contains products and published posts; contains no draft or archived slug.
Order flow, end to end	Add → cart → checkout → confirm. Verify stock decremented, order visible in <code>/admin/orders</code> , cancel restocks.
Oversell test	Two concurrent checkouts on the last unit. One must fail with <code>InsufficientStockError</code> ; stock must not go negative.
Authorization test	Call each admin action as a USER — expect a redirect, not a mutation. Request another user's <code>/orders/[id]</code> — expect 404.
Responsive pass	375 / 768 / 1024 / 1440. No horizontal body scroll; tables scroll inside their own container.
Keyboard pass	Tab the whole site. Skip link works, nav is traversable, focus visible everywhere, overlay traps and restores.
<code>axe DevTools + Lighthouse a11y</code>	Target 95+. Check contrast on 11px labels specifically.
Rich Results Test	Validate each JSON-LD type after Phase 7.
Reduced-motion pass	Enable OS reduced motion. Splash must not appear; all content must be visible with no tweens.

Env audit

Every key in `.env.example` present in Vercel. Confirm `NEXT_PUBLIC_SITE_URL` and `CRON_SECRET` — both missing locally today.

Every finding above was traced to a file and line in this repository on 3 September 2026, branch `main`. Where a claim could not be verified from the code, it is not in this report. No statistics, partner details, player facts or business figures were introduced – the project's data-integrity rule applies to its audit too.